AAI3008
Large Language Models
Transformer and BERT

Zhang Zhengchen
zhengchen.zhang@singaporetech.edu.sg

Review

- Byte Pair Encoding (BPE)
  - **To build subwords** from characters by **merging frequent pairs**
  - Handle rare words using subwords

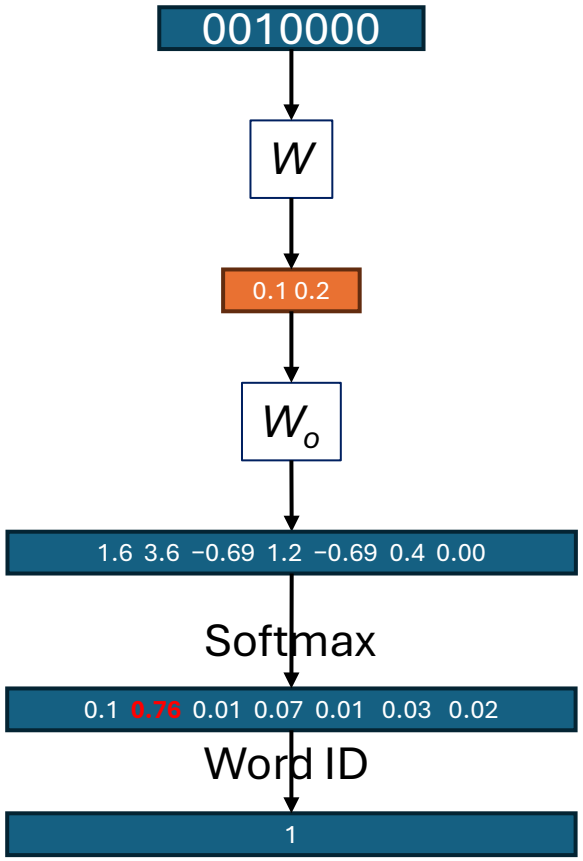
1. word → characters, build a vocabulary
2. Count the frequencies of bigrams
   - Higher order function in Python
   - *max* and dictionary
3. Merge the most frequently appeared one
   - Replace 'lo w' by 'low' in the vocabulary
   - Get a new vocabulary
4. Repeat 2 and 3 until some threshold is achieved

Algorithm 1 Learn BPE operations

```
import re, collections

def get_stats(vocab):
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
    for i in range(len(symbols)-1):
      pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
         'n e w e s t </w>':6, 'w i d e s t </w>':3}
num_merges = 10
for i in range(num_merges):
  pairs = get_stats(vocab)
  best = max(pairs, key=pairs.get)
  vocab = merge_vocab(best, vocab)
  print(best)
```

Review

- Word2Vec
  - Word ID to word embeddings
  - CBOW and Skip-Gram
  - Cosine Similarity
    - happy and glad: 0.74

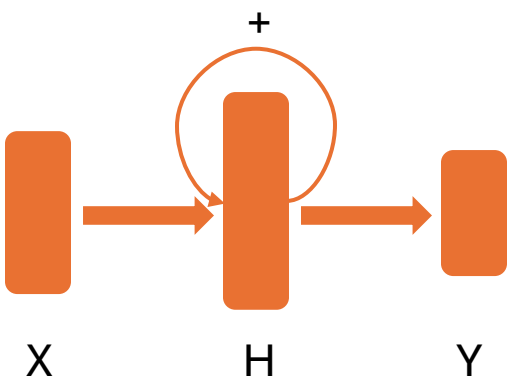


Outline

- Transformer
  - Attention
  - Encoder and Decoder
  - Normalization
  - Residual Connection
  - Positional Encoding

Attention is All You Need

- Transformer
  - A network architecture
  - Based solely on attention mechanisms
    - No recurrence, no convolutions



Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/pdf/1706.03762

Model Architecture

- Encoder
- Decoder

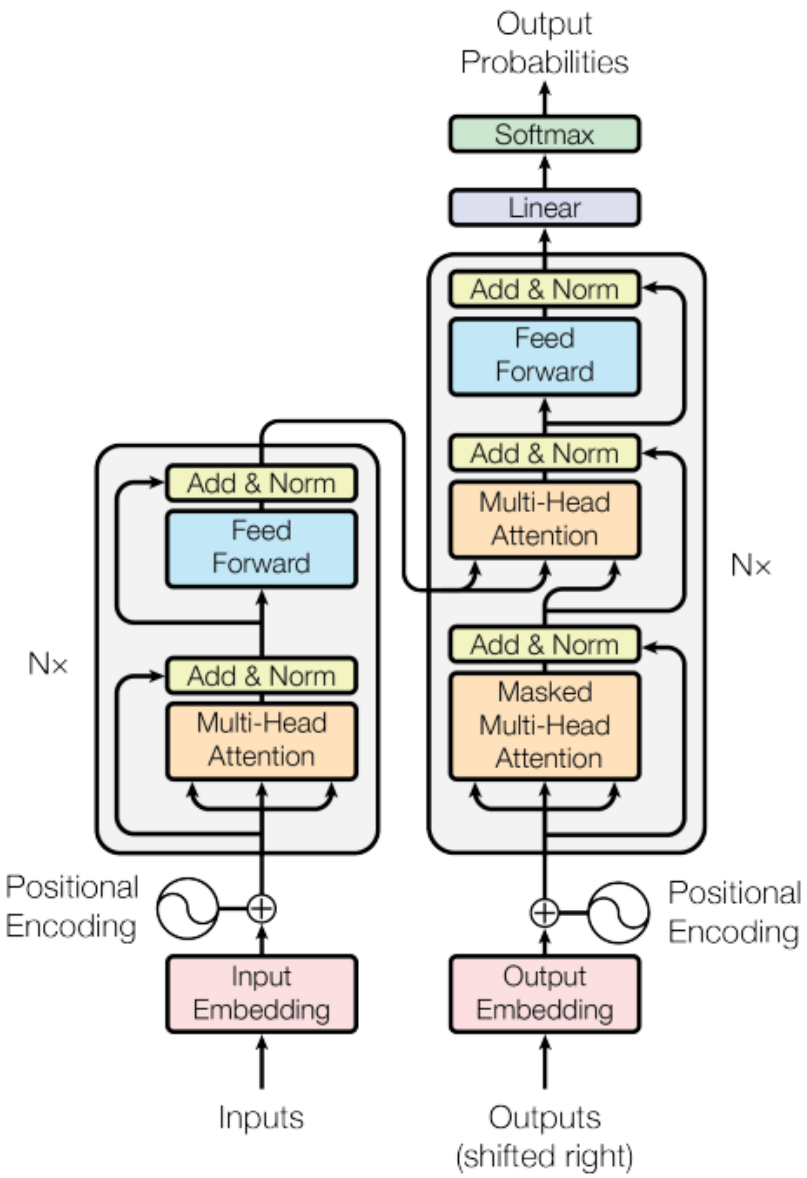


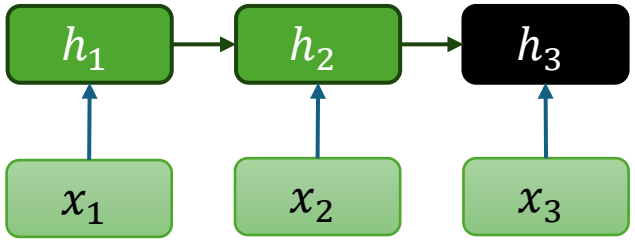
Figure 1: The Transformer - model architecture.

RNN based Encoder-Decoder

- Task
  - Sequence-to-Sequence problem
  - E.g. Machine Translation
    - The cat sat on the mat
    - Le chat s'est assis sur le tapis
    - <BOS> The cat sat on the mat <EOS>
- Key Characteristics
  - We can see the entire input sentence.
  - We do not know the output sentence in advance.
  - The model predicts the output tokens one by one.

RNN based Encoder-Decoder

- Encoder
  - An encoder is a component that takes an input and converts it into a meaningful internal representation.
  - The **final hidden state** of the model represents the context/summary of the whole input sequence
  - It acts as the initial hidden state of the decoder part of the model

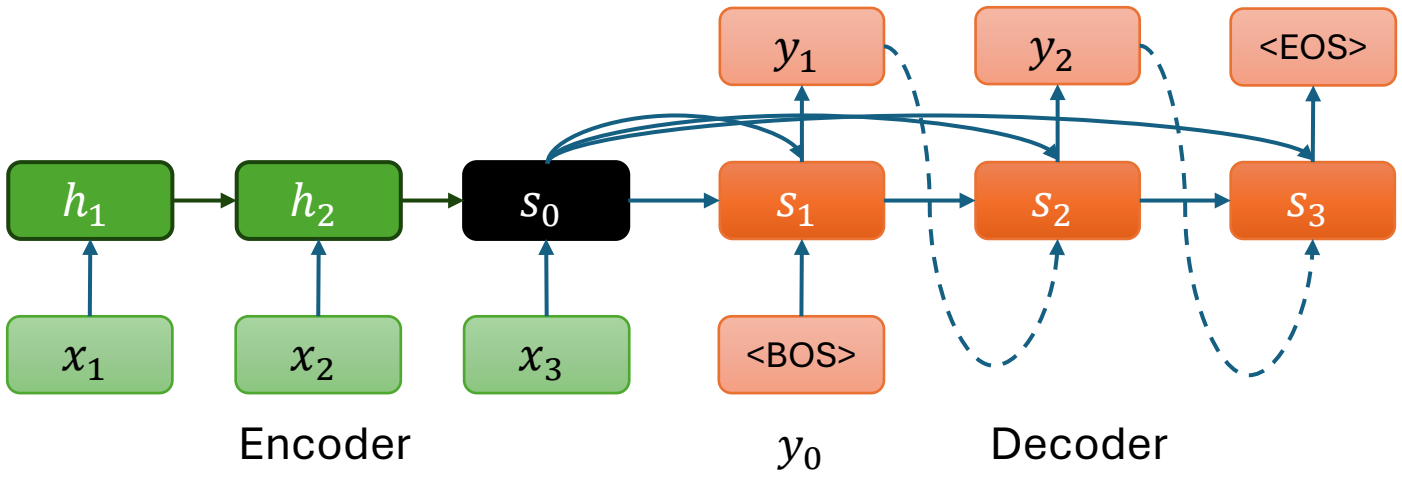


Encoder

RNN based Encoder-Decoder

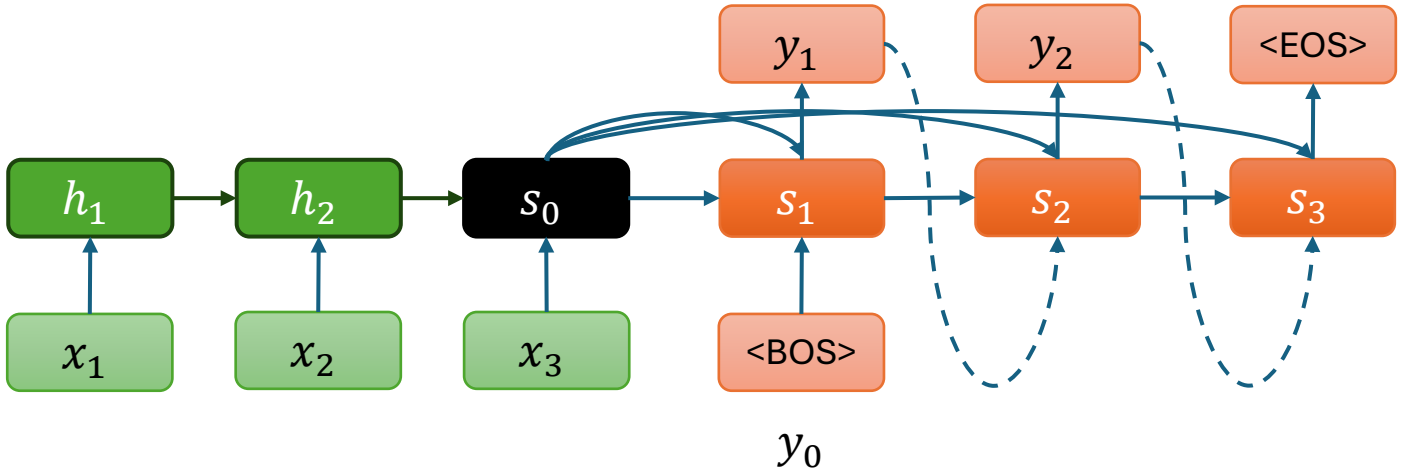
- Decoder
  - A decoder is a component that takes an internal representation and generates an output.

- Input to the Decoder
  - Predicted y_{t-1}
    - Starting output y_0: <BOS>
  - Initial hidden state $s_0 = c$
    - The final hidden state of encoder c
  - $s_i = g(W_s s_{i-1} + W_y y_{i-1} + W_c c + b1)$
  - $y_i = softmax(W_o s_i + b2)$



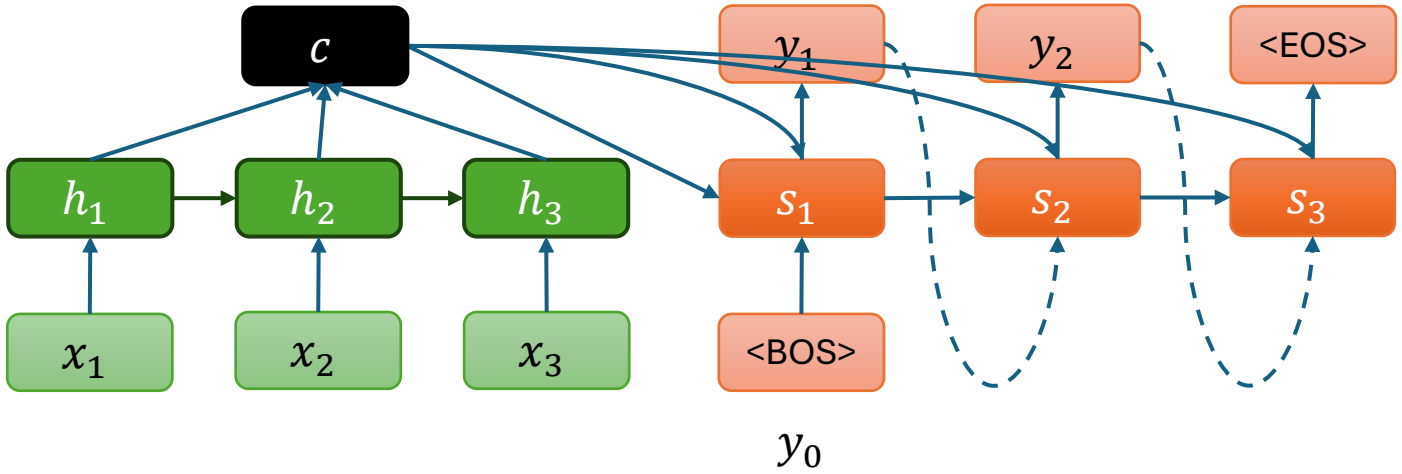
RNN based Encoder-Decoder

- The Encoder processes and **encodes the information** from all input elements.
- The Decoder utilizes this encoded information to **generate the target output** sequence.
- How does the decoding process begin?
  - It starts with a special token: <BOS> (Begin of Sequence).
- When does the decoding process stop?
  - It stops upon encountering the special token: <EOS> (End of Sequence).



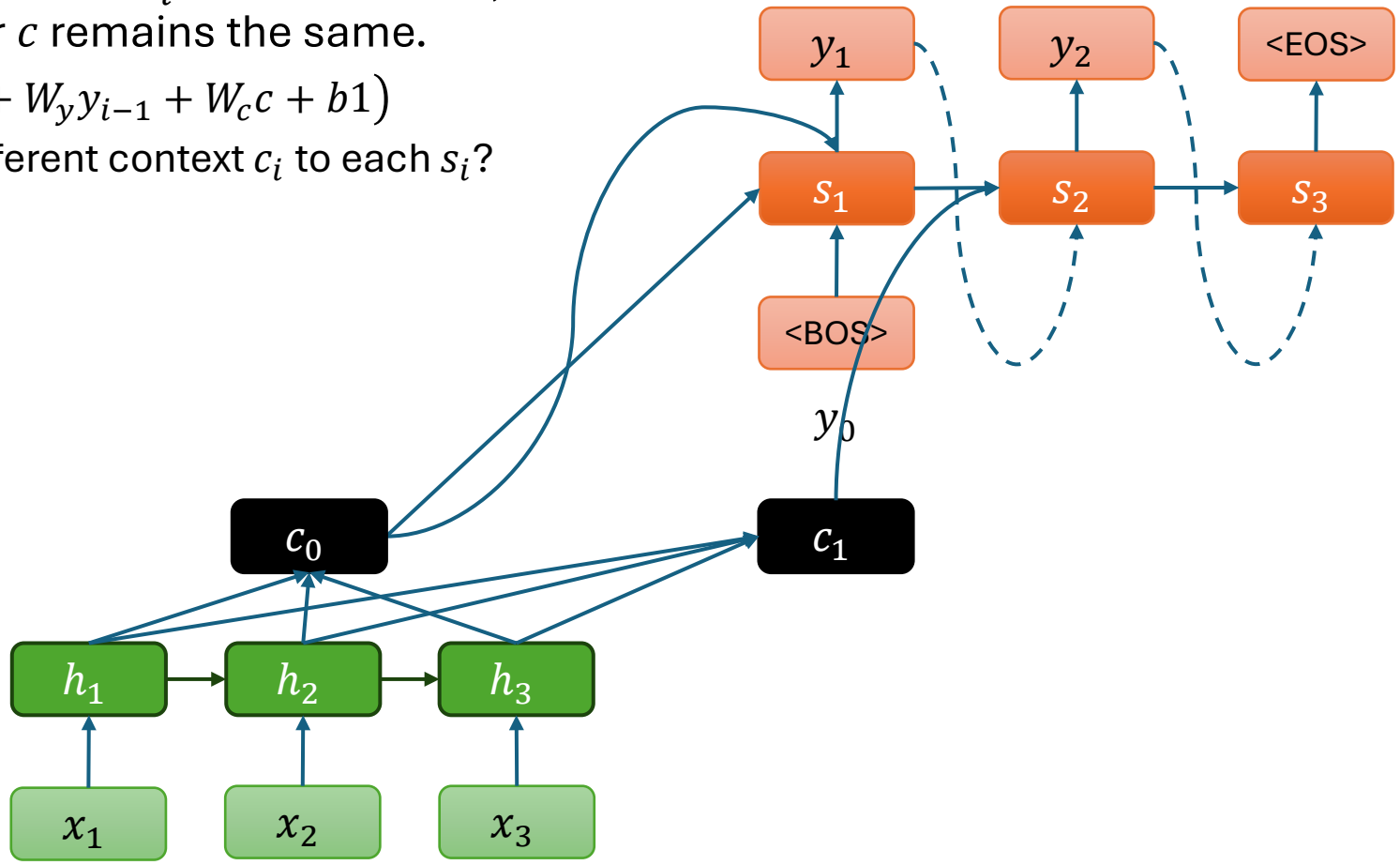
Problems of the context vector c

- Can the context vector contain all the information?
  - The context vector c is the final hidden state of the Encoder
  - However, this single vector may not capture all information, especially for long sequences
- Why don't we use all the hidden states h_j?
  - How?
  - By computing a weighted sum of all h_j into a single vector
    - $\boldsymbol{c} = \sum \alpha_j \boldsymbol{h}_j$



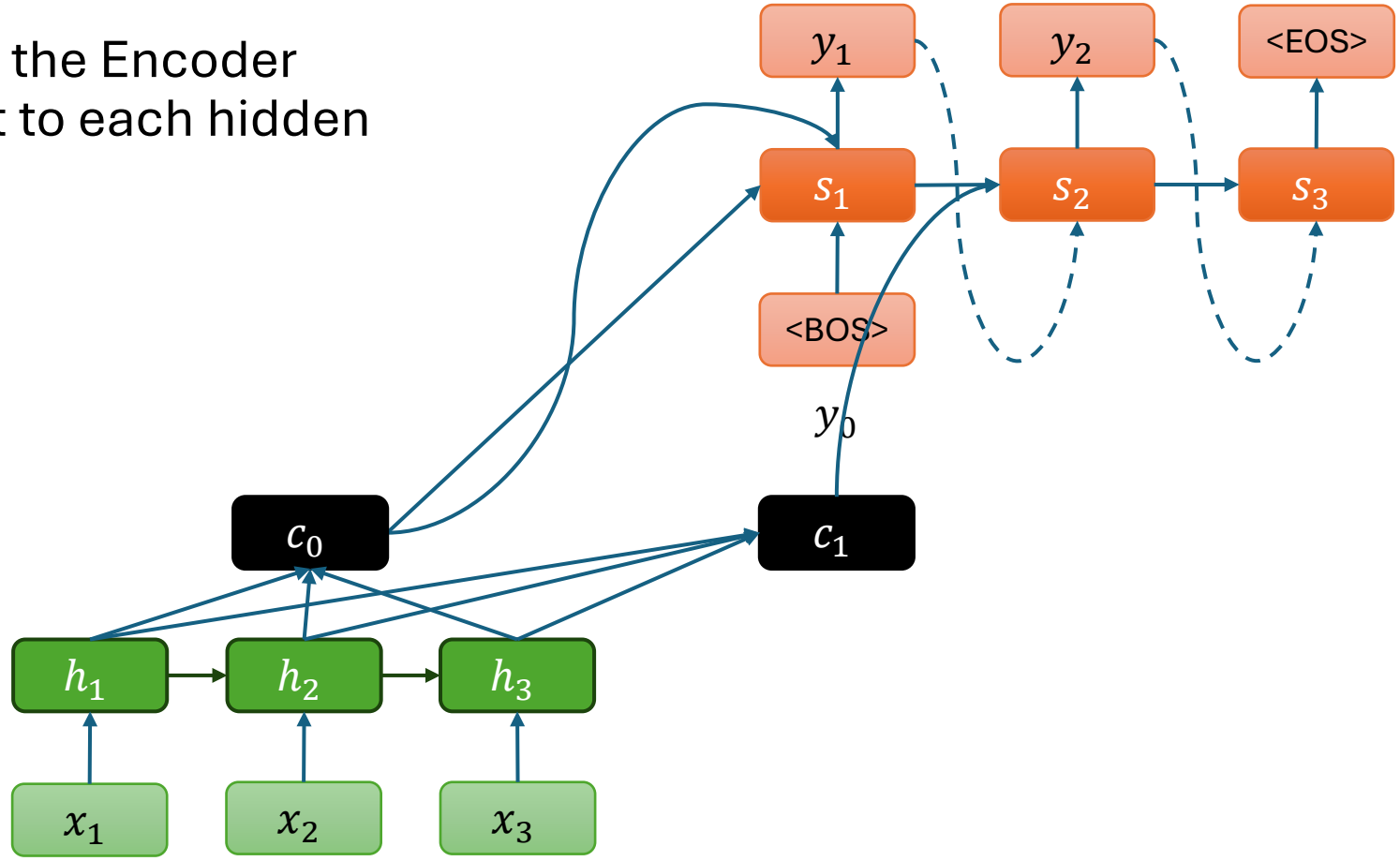
Problems of the context vector c

- For all the hidden states s_i in the Decoder, the context vector c remains the same.
  - $s_i = g(W_s s_{i-1} + W_y y_{i-1} + W_c c + b1)$
  - Can we give different context c_i to each s_i?
  - $\boldsymbol{c_0} = \sum \alpha_{0j} \boldsymbol{h_j}$
  - $\boldsymbol{c_1} = \sum \alpha_{1j} \boldsymbol{h_j}$
  - $\boldsymbol{c_i} = \sum \alpha_{ij} \boldsymbol{h_j}$



Assigning Unique Context c_i to Each Decoder Step

- $\boldsymbol{c_i} = \sum \alpha_{ij} \boldsymbol{h_j}$
  - Use all hidden states of the Encoder
  - Assign a unique context to each hidden state in the Decoder
- How to determine α_{ij}?



Bahdanau Attention

$$p(y_i|y_1,\ldots,y_{i-1},\mathbf{x}) = g(y_{i-1}, s_i, c_i),$$

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})},$$

$$e_{ij} = a(s_{i-1}, h_j)$$

$$a(s_{i-1}, h_j) = v_a^\top \tanh \boxed{(W_a s_{i-1} + U_a h_j)}$$

Additive attention

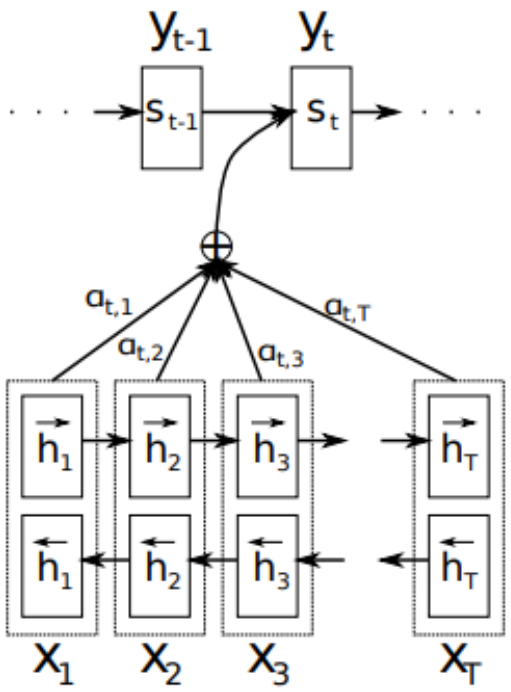


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.

Bahdanau D. Neural machine translation by jointly learning to align and translate[J]. arXiv preprint arXiv:1409.0473, 2014. https://arxiv.org/pdf/1409.0473

Attention is all you need

- An attention function can be described as
  - **mapping a query and a set of key-value pairs to an output**
  - where the query, keys, values, and output are all vectors.

- The **output is computed as a weighted sum of the values**
  - where the **weight** assigned to each value is computed by a **compatibility function of the query with the corresponding key**.
  - α_{ij} represents the relationship between h_j and s_{i-1}

Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/pdf/1706.03762

Attention is all you need

- **An Intuitive View of Attention (Learning / Project Context)**
- **Query**: the current problem you are trying to solve
  - e.g., a bug, a design decision, or a concept you are unsure about
- **Key**: concepts or semantic labels
  - e.g., Tokenizer, RAG, Evaluation, Prompt Design
  - used to identify *which content might be relevant*
- **Value**: the concrete content
  - e.g., slides, example code, implementation details, explanations
- **What Attention Does**:
  - given a query, it finds the most relevant concepts (keys)
  - and retrieves only the corresponding content (values)
- **Core Idea**:
  - you do not review everything every time
  - you focus only on what is most relevant to the current problem

Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/pdf/1706.03762

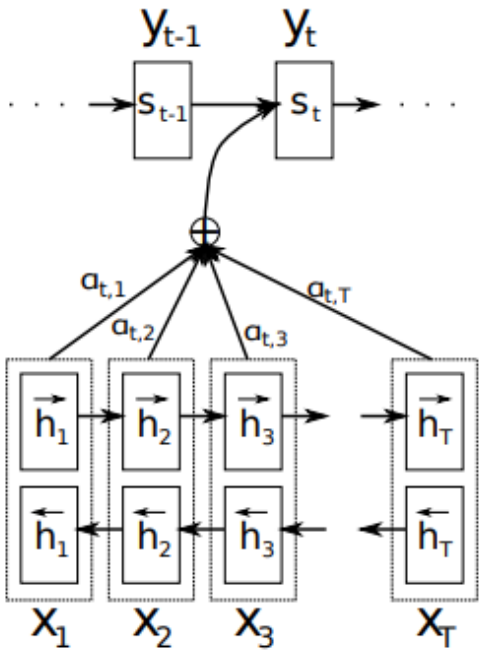
Attention is all you need

- An attention function can be described as
  - **mapping a query and a set of key-value pairs to an output**
  - where the query, keys, values, and output are all vectors.

  - Who is the query? Who is the key?
    - The query is s_{i-1}
    - The keys are the hidden states h_j

  - The **output is computed as a weighted sum of the values**
    - Who is the output? Who is the value?
    - $\boldsymbol{c_i} = \sum \alpha_{ij} \boldsymbol{h_j}$

  - where the **weight** assigned to each value is computed by a **compatibility function of the query with the corresponding key**.
    - What is the weight?
    - Compatibility function



$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})},$$

$$e_{ij} = a(s_{i-1}, h_j)$$

$$a(s_{i-1}, h_j) = v_a^\top \tanh(W_a s_{i-1} + U_a h_j),$$

Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/pdf/1706.03762

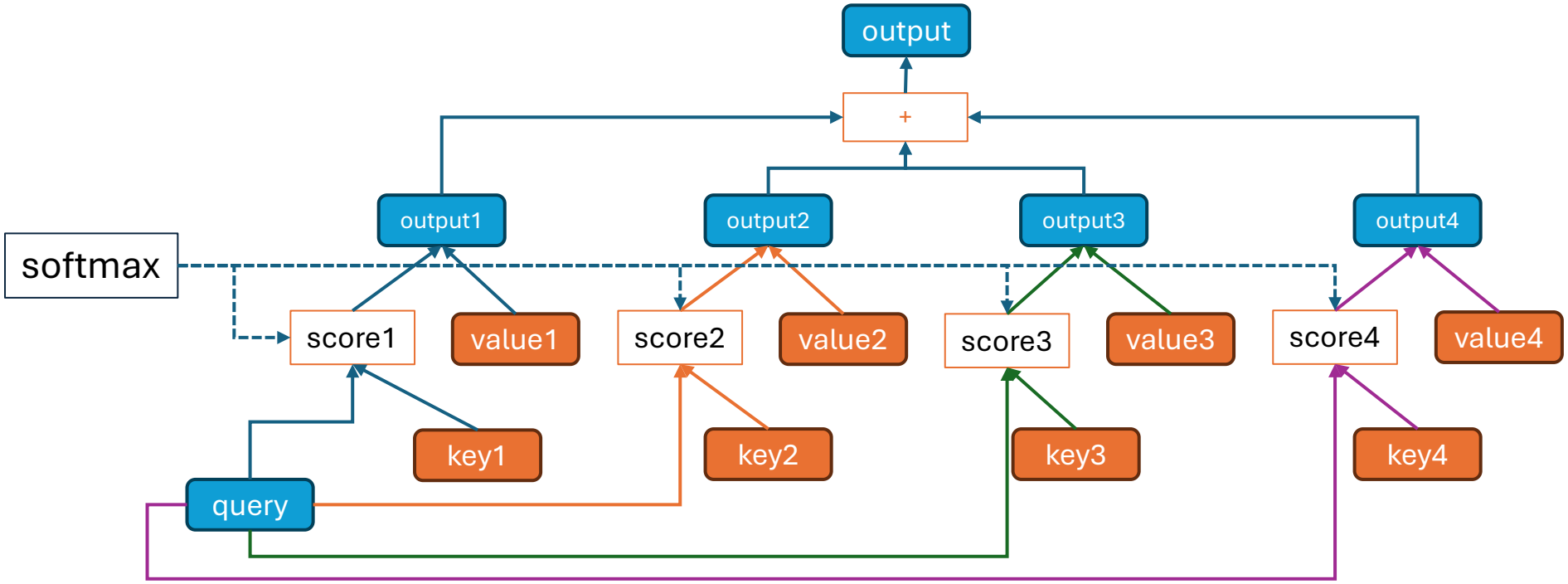
Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- mapping a query and a set of key-value pairs to an output
- output is computed as a weighted sum of the values
- where the weight is computed by a compatibility function of the query with the corresponding key.

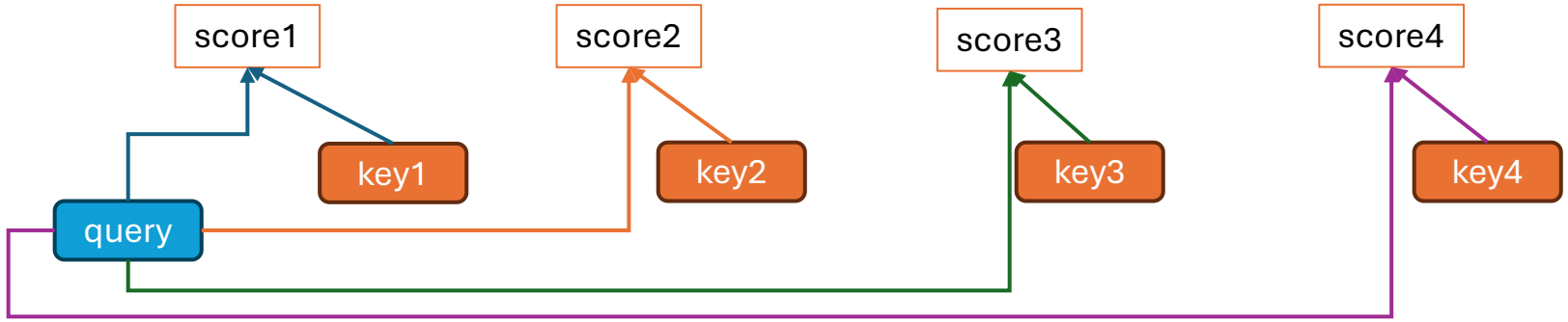
Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/pdf/1706.03762

An Example


output
+
output1
output2
output3
output4
softmax
score1
value1
score2
value2
score3
value3
score4
value4
key1
key2
key3
key4
query

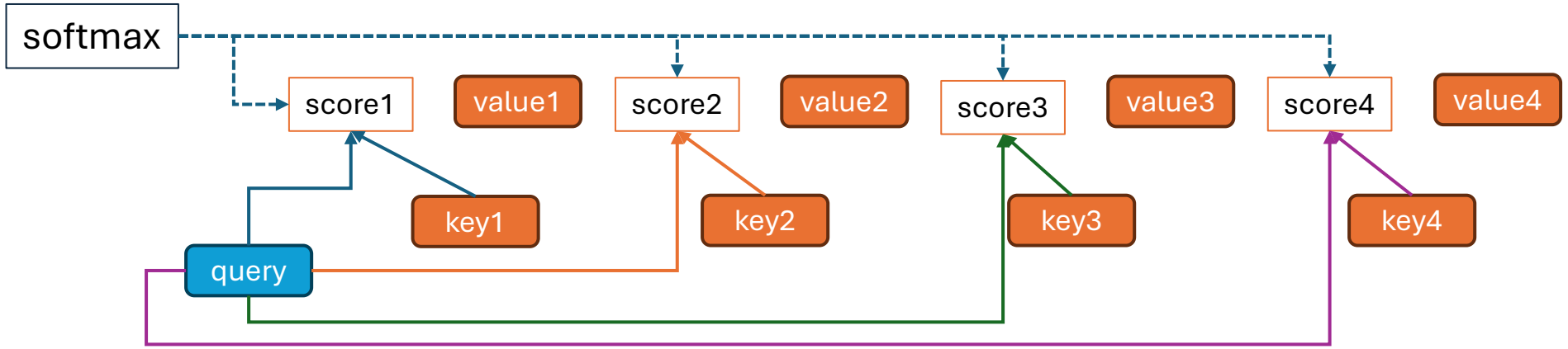
Dot-Product Attention

- Given a query, the model compares it with all keys.
  - The dot product measures how similar the query is to each key.
  - $dot\ product(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{d} v_i w_i = v_1 w_1 + v_2 w_2 + \cdots + v_d w_d$
  - Higher dot-product values indicate stronger relevance.
- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- $\boldsymbol{q} \in R^d$, $\boldsymbol{key} \in R^d$, $d = 3$. We have 4 keys → $K \in R^{4 \times d}$
- $score = qK^T = [q_1\ q_2\ q_3] \begin{bmatrix} k_{11} & \dots & k_{41} \\ k_{12} & \dots & k_{42} \\ k_{13} & & k_{43} \end{bmatrix} = [\alpha_{00}\ \alpha_{01}\ \alpha_{02}\ \alpha_{03}]$



Dot-Product Attention

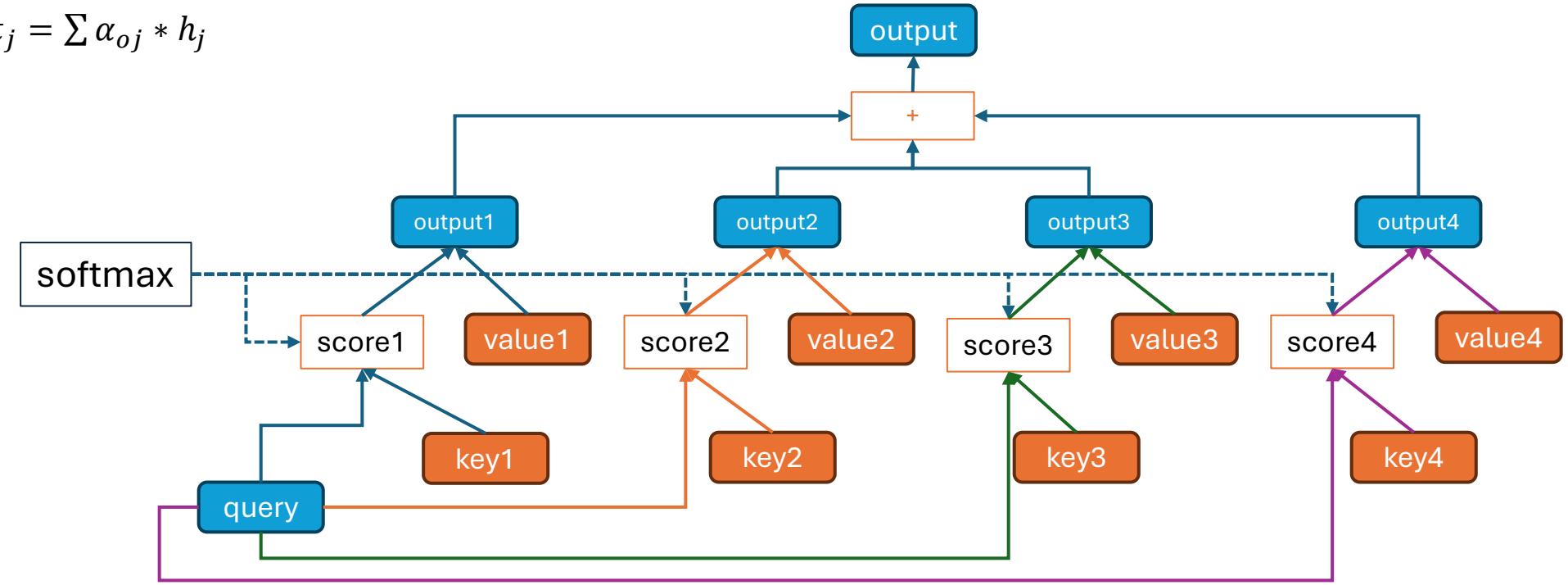
- $score = qK^T = [\alpha_{00}, \alpha_{01}, \alpha_{02}, \alpha_{03}]$
- Softmax converts the values of the scores between 0 and 1, and their sum equals 1
- $\boldsymbol{\alpha} = softmax(qK^T)$



Dot-Product Attention

- The **output is computed as a weighted sum of the values**
- In this case, keys and values are all h_j
- $output1 = \alpha_{00} * h_0 = 0.1 * [1,1,1] = [0.1,0.1,0.1]$
- $output2 = \alpha_{01} * h_1 = 0.2 * [2,2,2] = [0.4,0.4,0.4]$
- $output = \sum output_j = \sum \alpha_{oj} * h_j$

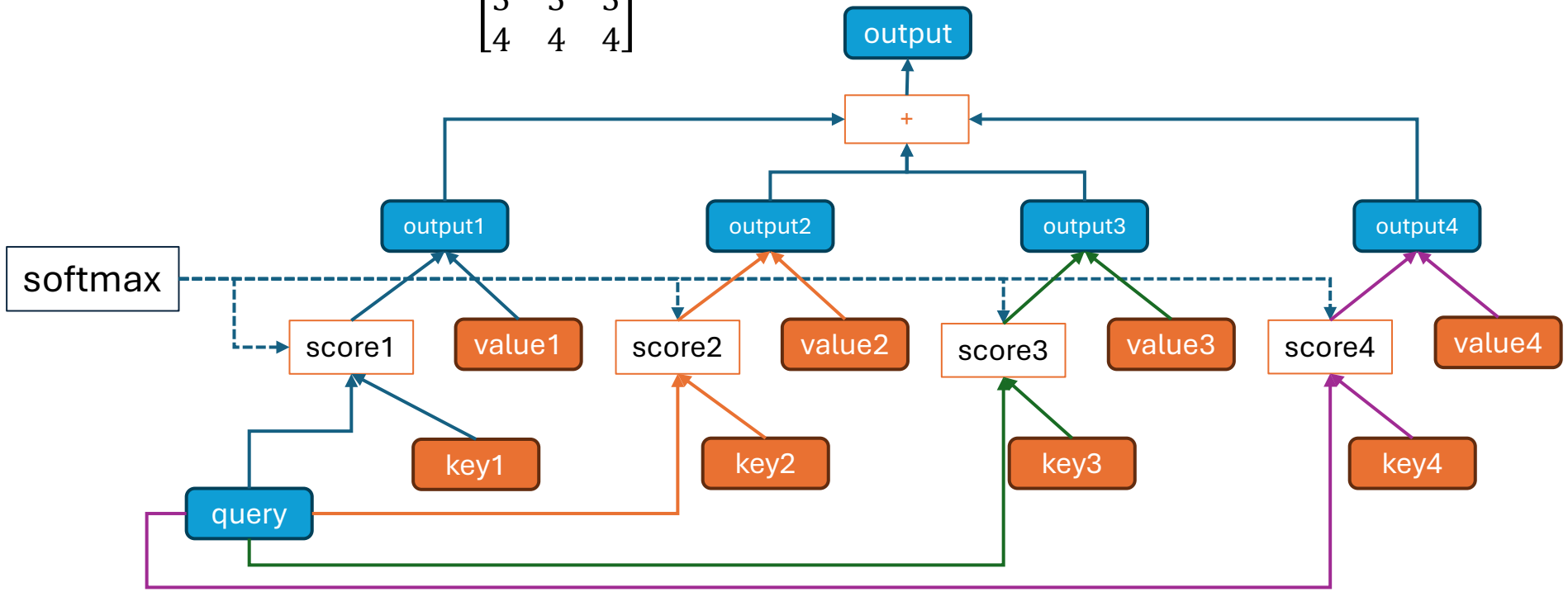
$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$



Dot-Product Attention

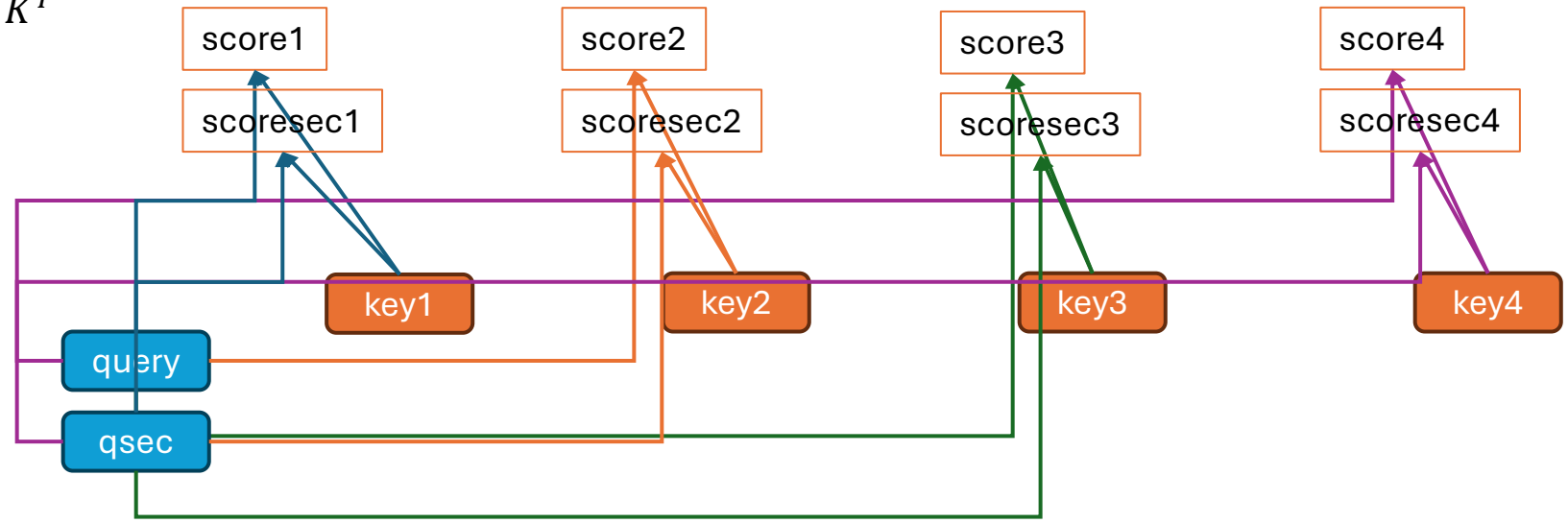
- The **output is computed as a weighted sum of the values**
- $output = \sum output_j = \sum \alpha_{oj} * h_j$
- $output = \boldsymbol{\alpha} V = softmax(qK^T)V = [0.1\ 0.2\ 0.3\ 0.4]\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \\ 4 & 4 & 4 \end{bmatrix} = [3,3,3]$
- $V \in R^{4\times d}$

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$



Dot-Product Attention

- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- $output = \boldsymbol{\alpha}V = softmax(qK^T)V$
- How may queries do we have? Only 1.
- What if there are 2 queries?
- $\boldsymbol{qsec} \in R^d, scoresec1 = dot\ product(\boldsymbol{qsec}, \boldsymbol{key})$
- $scoresec = qsec \cdot K^T$



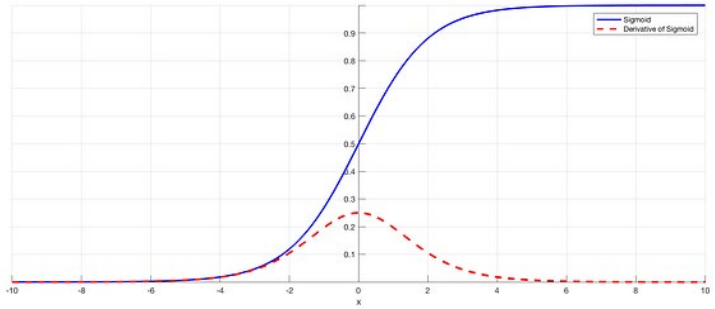
Dot-Product Attention

- $score = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} K^T = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{bmatrix}^T = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \begin{bmatrix} k_1 & k_2 & k_3 & k_4 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.21 & 0.22 & 0.23 & 0.24 \end{bmatrix}$
- $Output = \boldsymbol{\alpha V} = softmax(score) \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = softmax(QK^T)V$
- $Attention(Q, K, V) = softmax(QK^T)V$
  - If the input of the softmax function is a **matrix**, Softmax is applied **independently to each row** of the matrix by default.

Scaled Dot-Product Attention

- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- d_k is the dimension of keys and values
- Larger values of d_k affects the performance of dot product attention
- We **suspect** that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients.

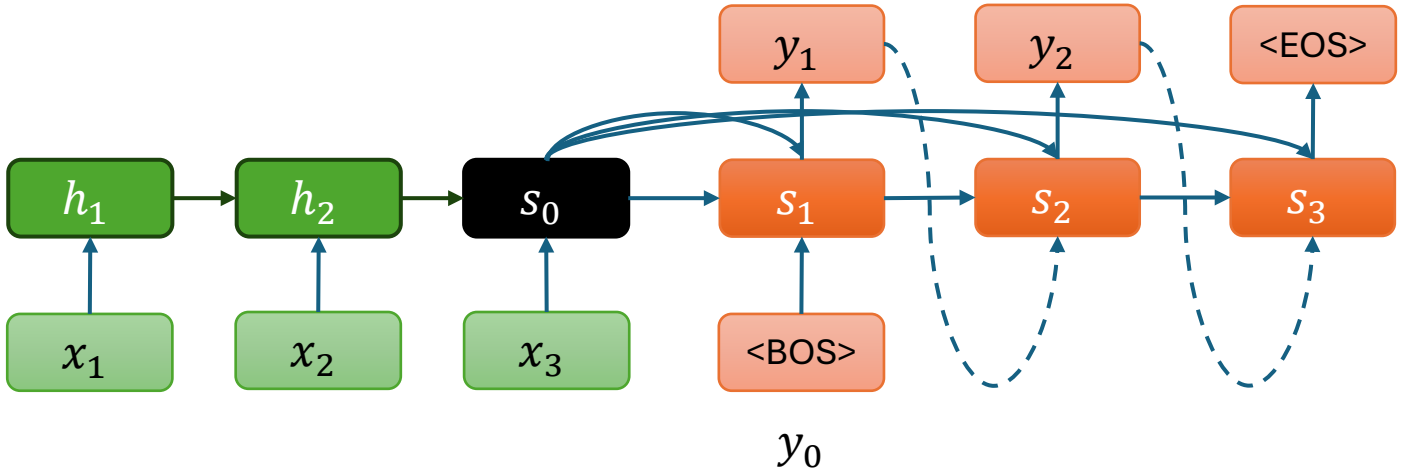
- To counteract this effect, we scale the dot products by $\sqrt{d_k}$



sigmoid function and its derivative
Sigmoid is a two-class softmax.

Self-Attention

- RNN based Encoder-Decoder
- Query
  - Hidden states of the Decoder
- Key and Value
  - Hidden states of the Encoder



Self-Attention

- Query, Key and Value
  - The input sequence X is linearly transformed to produce the Query, Key, and Value matrices
  - Using three independent learnable weight matrices W_Q, W_K, and W_V
    - $Q = XW_Q$
    - $K = XW_K$
    - $V = XW_V$

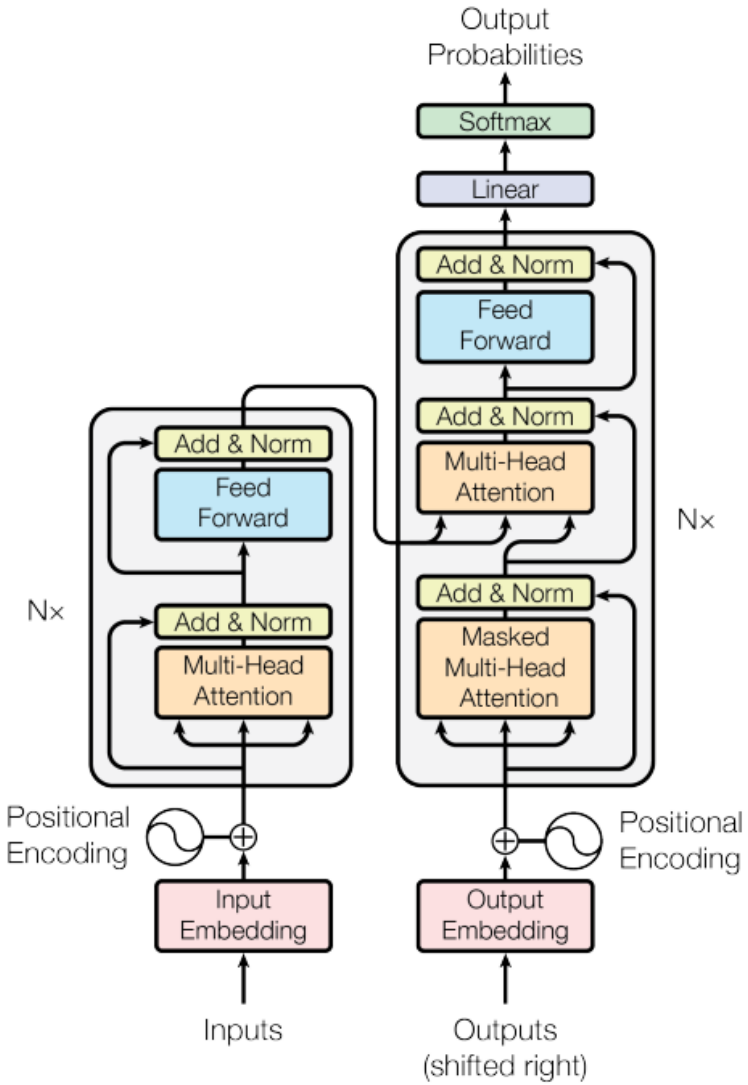


Figure 1: The Transformer - model architecture.

Dot-Product Attention

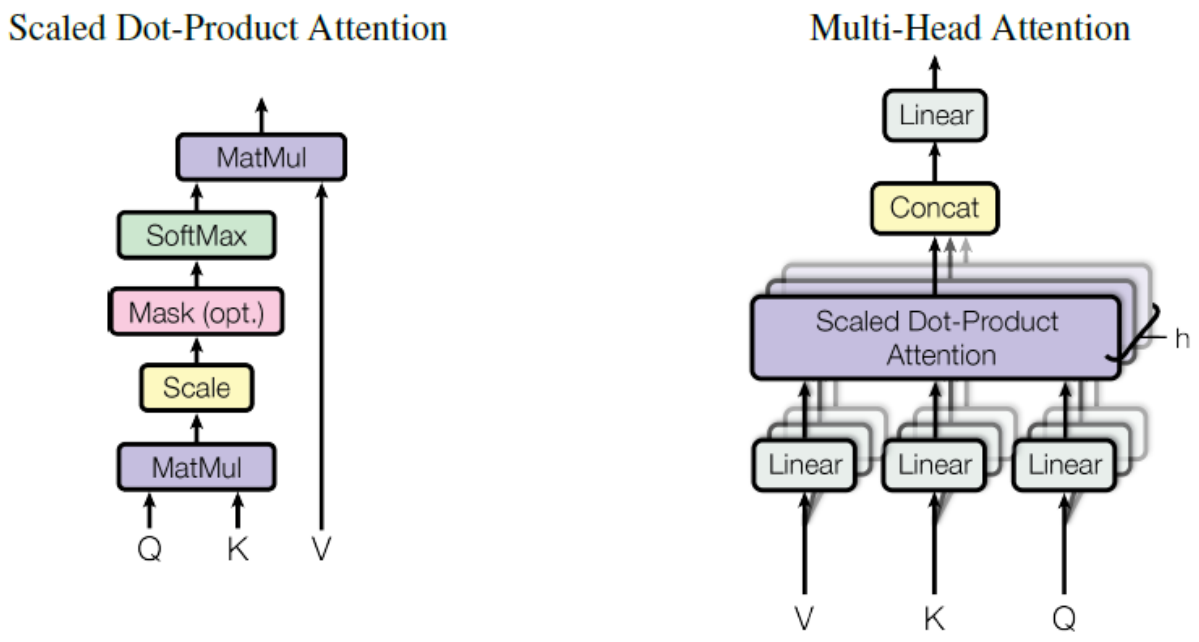
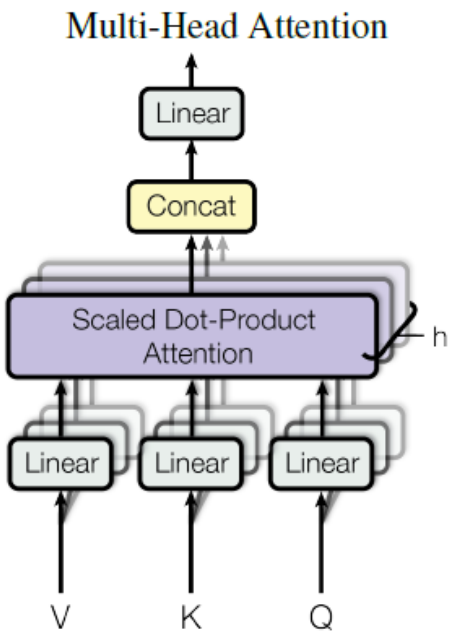


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Multi Head Attention

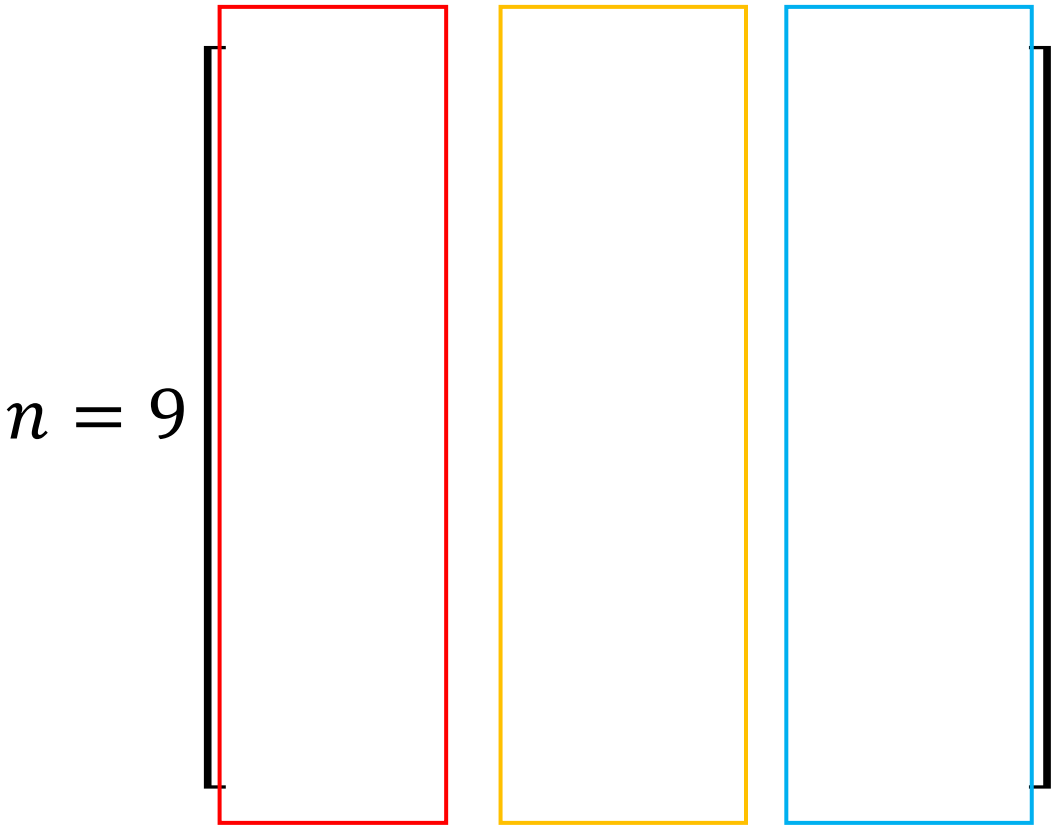
- It's **beneficial** to linearly project the queries, keys and values $\boldsymbol{h}$ **times** with **different**, learned linear projections to d_k, d_k, and d_v dimensions, respectively.
  - Before: Map to 512 dimension
  - After: Map to 64 dimension for 8 times
  - In my understanding, we tell the model these 64 dimensions learn different features

- Head number $h = 8, d_k = d_v = \frac{d_{model}}{h} = 64$
- These are concatenated and once again projected, resulting in the final values.



$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, ..., \mathrm{head}_\mathrm{h})W^O$$
$$\text{where } \mathrm{head}_\mathrm{i} = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.



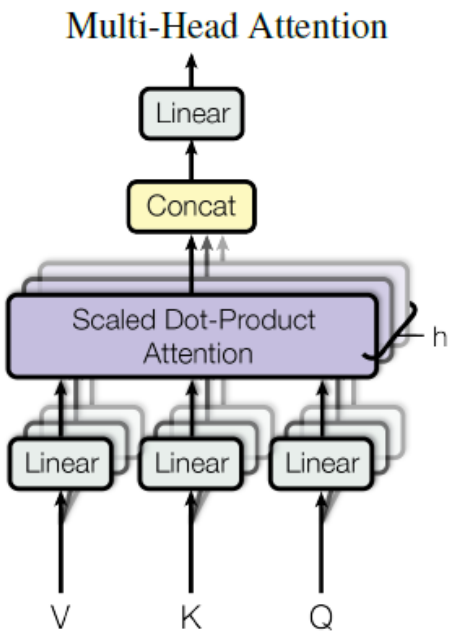
$$d_{model} = 6$$
$$h = 3$$
$$d_k = d_v = 2$$

Multi Head Attention

- Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions.
- With a single attention head, this diversity is lost due to averaging.

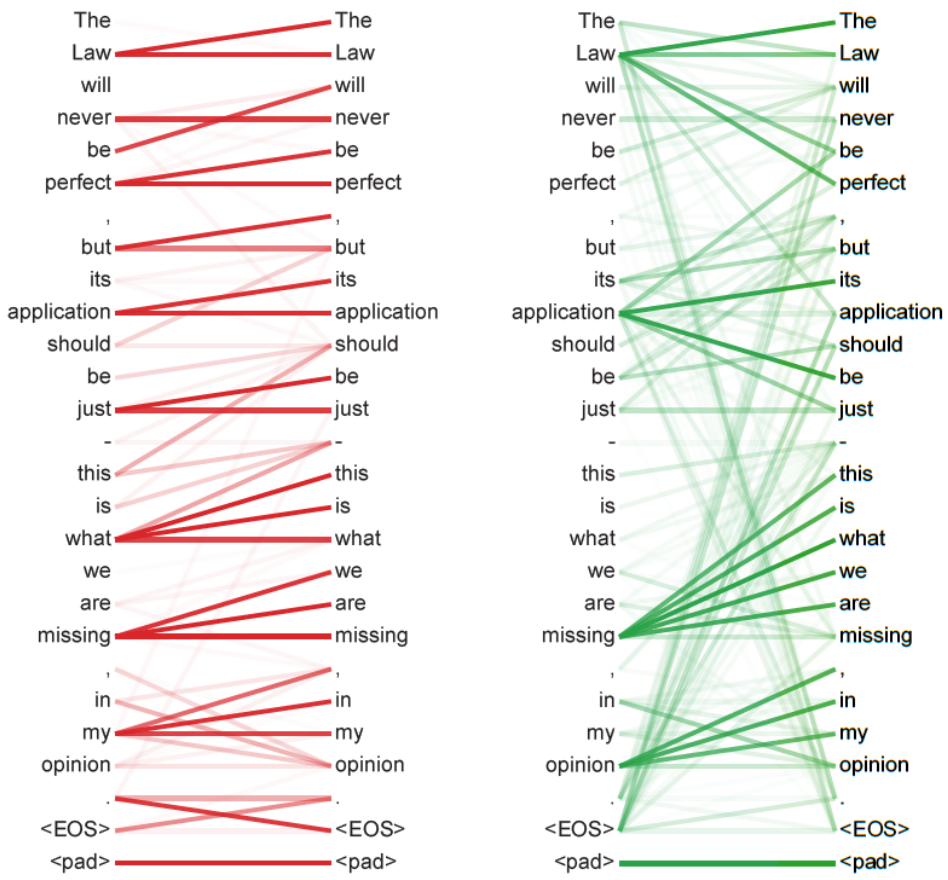
$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, ..., \mathrm{head}_\mathrm{h})W^O$$
$$\text{where } \mathrm{head}_\mathrm{i} = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.



Multi Head Attention

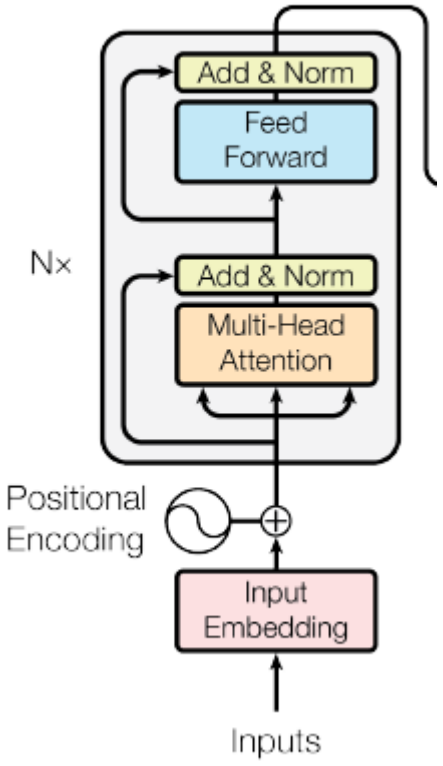
- Example
  - From two different heads from the encoder self-attention at layer 5 of 6.
  - The heads clearly learned to perform different tasks.
  - Different positions



Encoder

- Each layer has two sub-layers
  - The first is a **multi-head self-attention mechanism**
  - The second is a simple, **position-wise fully connected feed-forward network**.
  - A **residual connection** around each of the two sub-layers
  - Followed by **layer normalization**

- N = 6



Attention

- The encoder contains self-attention layers.
  - All of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder.
  - Each position in the encoder can **attend to all positions**.

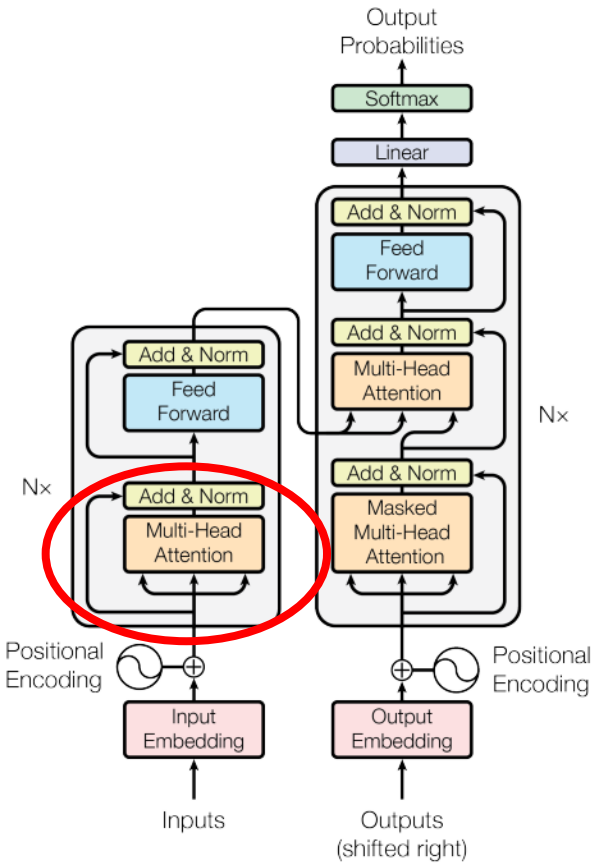
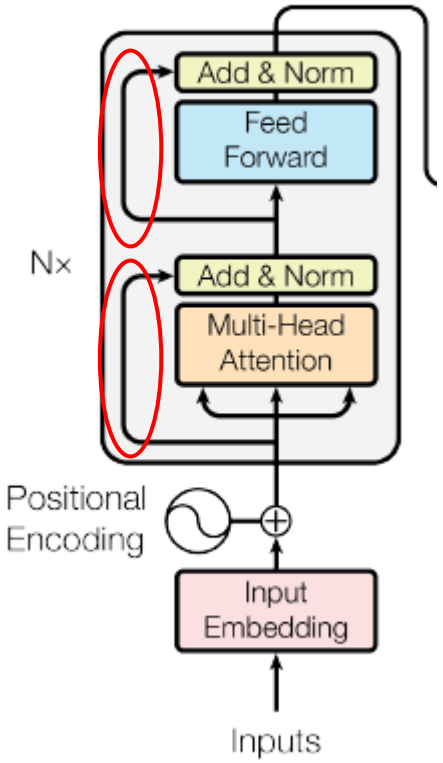


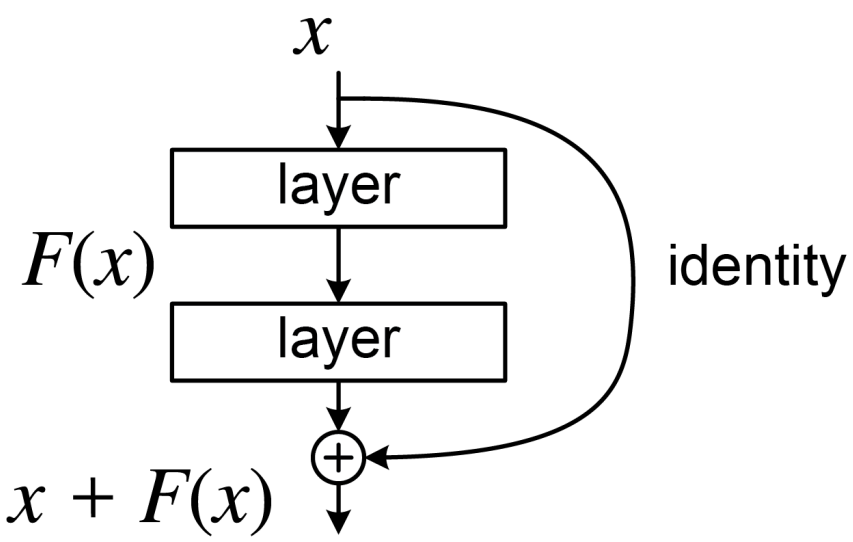
Figure 1: The Transformer - model architecture.

Residual Connections

- Motivation
  - Deep models may forget previously learned information.
  - As layers increase, useful signals can be weakened or distorted.
- Key Idea
  - Residual connections allow information to flow directly across layers.
  - Each layer learns a small refinement instead of a completely new representation.
- Formulation
  - $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$
  - $\mathbf{x}$:input to the layer
  - $F(\mathbf{x})$:newly learned transformation
- Intuition
  - If the new layer is not useful, the model can fall back to the original input.
  - This helps prevent the model from forgetting earlier representations.



Residual Connections



aerial roots of a banyan tree

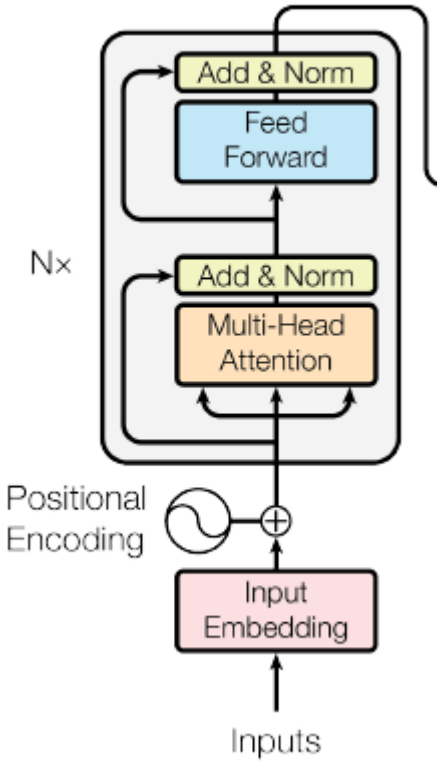
https://en.wikipedia.org/wiki/Residual_neural_network
https://en.wikipedia.org/wiki/Aerial_root

Normalization

- Normalization rescales values into a controllable range.
- This prevents values from becoming too large or too small.

$$\hat{x} = \frac{x - \mu}{\sigma}$$

- We subtract the mean and divide by the standard deviation.
- This keeps values centered and well-scaled.



Normalization

Original values

Suppose we have three values:

$$x = [10,\ 12,\ 14]$$

Step 1: Compute the mean

$$\mu = \frac{10 + 12 + 14}{3} = 12$$

Step 2: Compute the standard deviation

$$\sigma = \sqrt{\frac{(10-12)^2 + (12-12)^2 + (14-12)^2}{3}} = \sqrt{\frac{4+0+4}{3}} \approx 1.63$$

Step 3: Normalize

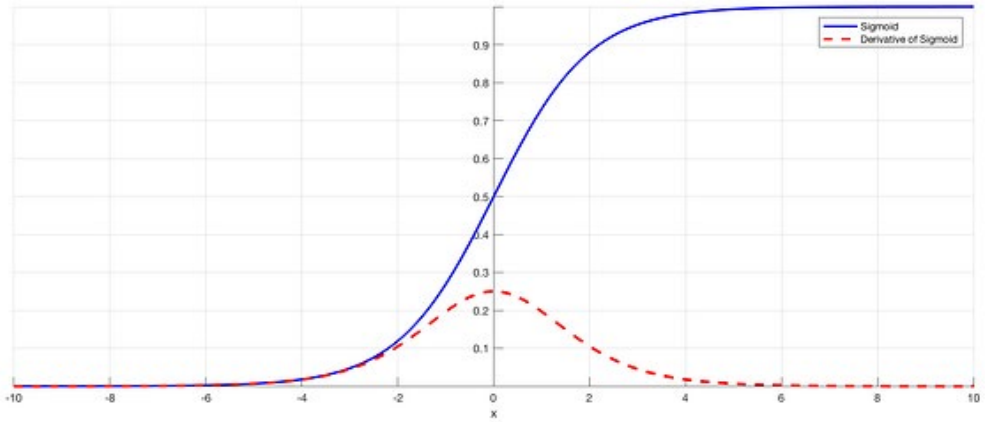
$$\hat{x} = \frac{x - \mu}{\sigma}$$

So we get:

$$\hat{x} \approx [-1.22,\ 0,\ 1.22]$$

Normalization

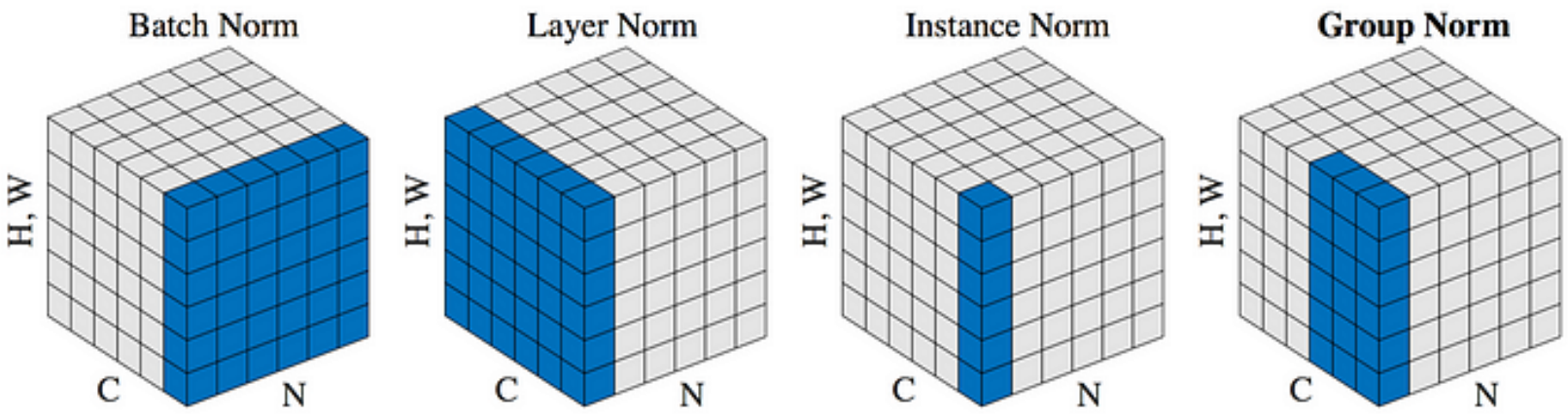
- Why does scale matter?
  - Consider an activation function like sigmoid.
  - If the input is too large or too small, the gradient becomes close to zero.
  - This makes learning very slow.
  - In deep models, values are repeatedly transformed and multiplied.
  - Without control, they may explode or vanish.
  - Errors and gradients can easily go out of control.



sigmoid function and its derivative

Layer normalization

- When we have batched data, especially batched sequence data, variations can come from different dimensions.



https://www.youtube.com/watch?v=1JmZ5idFcVI

Position-wise fully connected feed-forward network

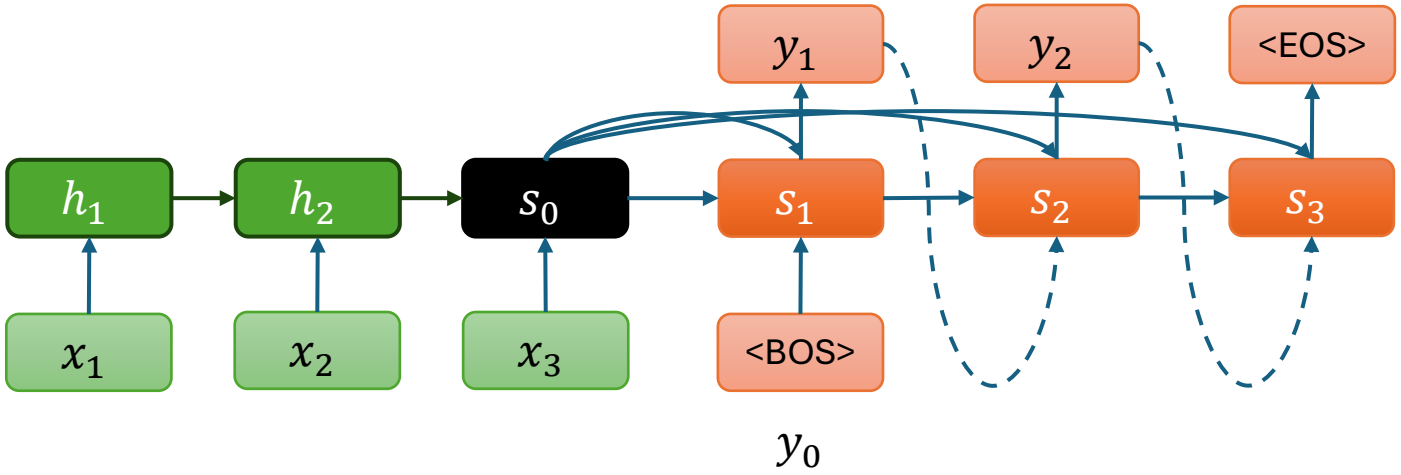
- Fully connected feed-forward network
- It is applied to each position separately and identically
- The linear transformations are the same across different positions
- They use different parameters from layer to layer

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Decoder

- In the **encoder**, we have access to **all the tokens** of the input sequence **simultaneously**.

- In the **decoder**, we generate the output sequence step by step, **one token at a time**.



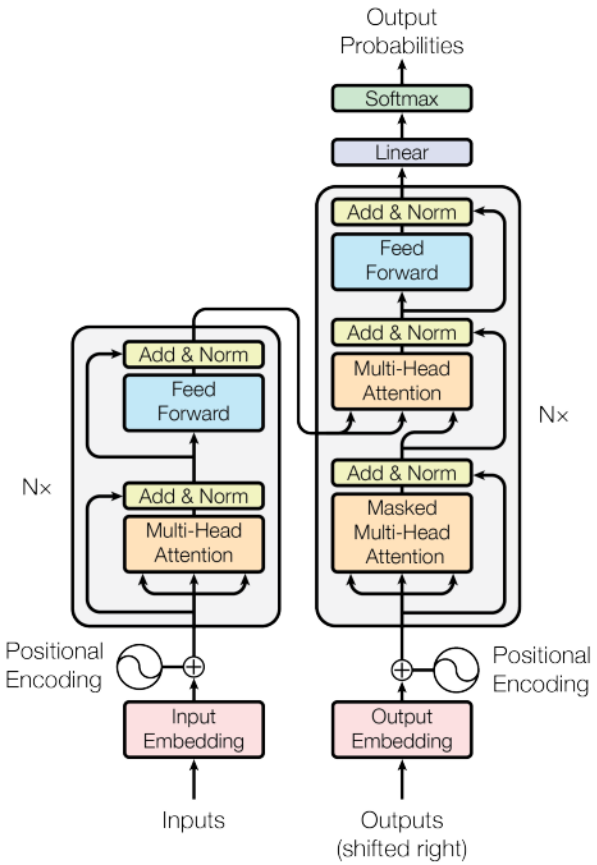


Figure 1: The Transformer - model architecture.

Masked self-attention layers in the decoder

- We need to prevent leftward information flow in the decoder to preserve the auto-regressive property.
  - For example, if you have only generated $[y_1, y_2]$, the model cannot access or "see" the values of $[y_3, y_4]$
- Mask out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

- Formula (with mask M):

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

  - M is the triangular mask ensuring causal behavior.

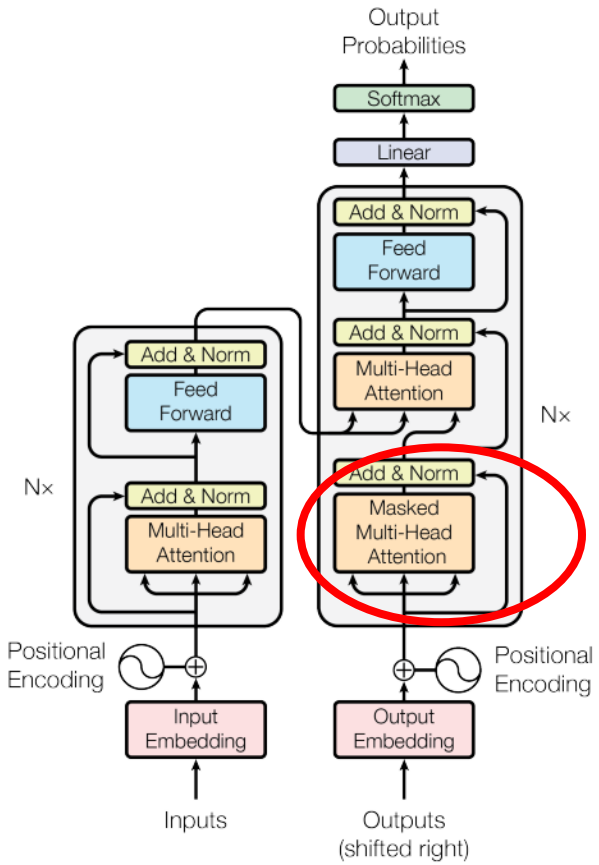


Figure 1: The Transformer - model architecture.

Applications of Attention in Transformer

- Encoder-decoder attention
  - The **queries** come from the previous **decoder** layer
  - The memory **keys** and **values** come from the output of the **encoder**
  - This allows every position in the decoder to attend **over all positions** in the input sequence

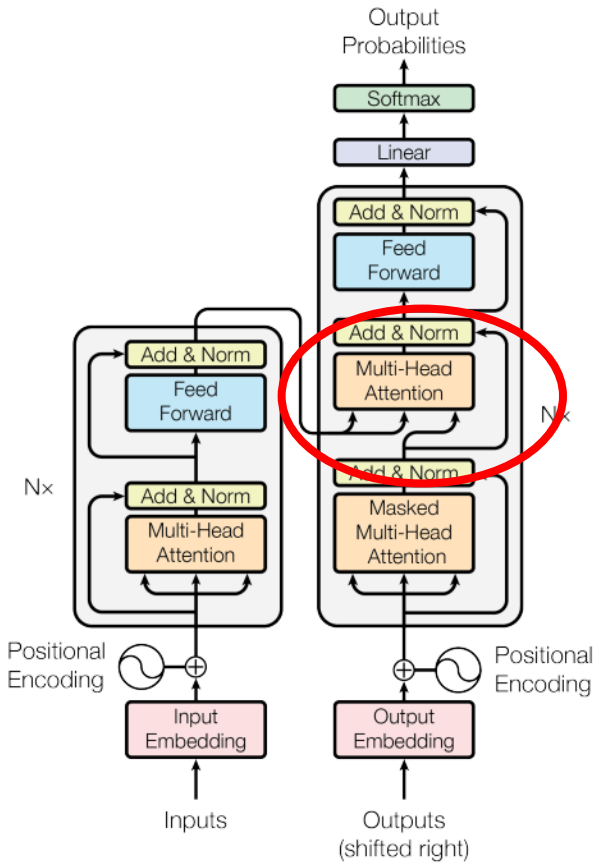


Figure 1: The Transformer - model architecture.

Embeddings

- We use **learned embeddings** to convert the input tokens and output tokens to vectors of dimension d_{model}.
- We also use the usual **learned linear transformation and softmax function** to convert the decoder output to predicted next-token probabilities.
- In our model, we **share the same weight matrix** between the two embedding layers and the pre-softmax linear transformation

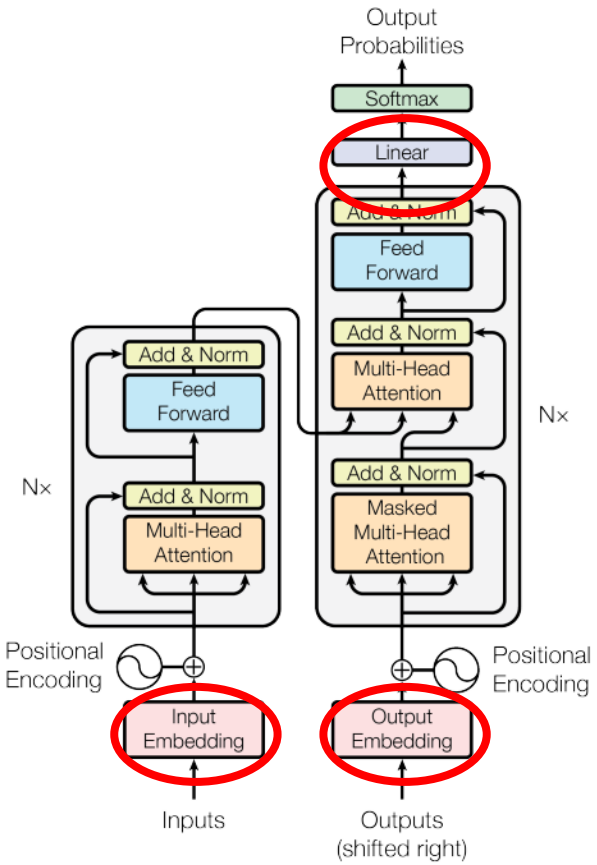


Figure 1: The Transformer - model architecture.

Positional Encoding

- Why do we need positional encoding?
  - Self-attention itself is **order-agnostic**
  - To use the order of the sequence
  - '*Book a book in the library*'

- Each token embedding is **augmented** with a position-dependent vector:
- Input *to Transformer* = *Token Embedding* + *Positional Encoding*
- Each position is represented by a vector, not just a single number.

Positional Encoding

- For each token position *pos*, the Transformer creates a positional embedding vector
$$\mathrm{PE}(pos) \in \mathbb{R}^{d_{model}}$$
- This vector has the same dimensionality as the token embedding, so it can be added element-wise.

- How is this vector constructed?
  - Let i range from 0 to $\frac{d_{model}}{2} - 1$, where i indexes the dimensions of the embedding vector.
  - Each dimension of the positional embedding is defined as:
  - Even dimensions (2i)
  - Odd dimensions (2i + 1)

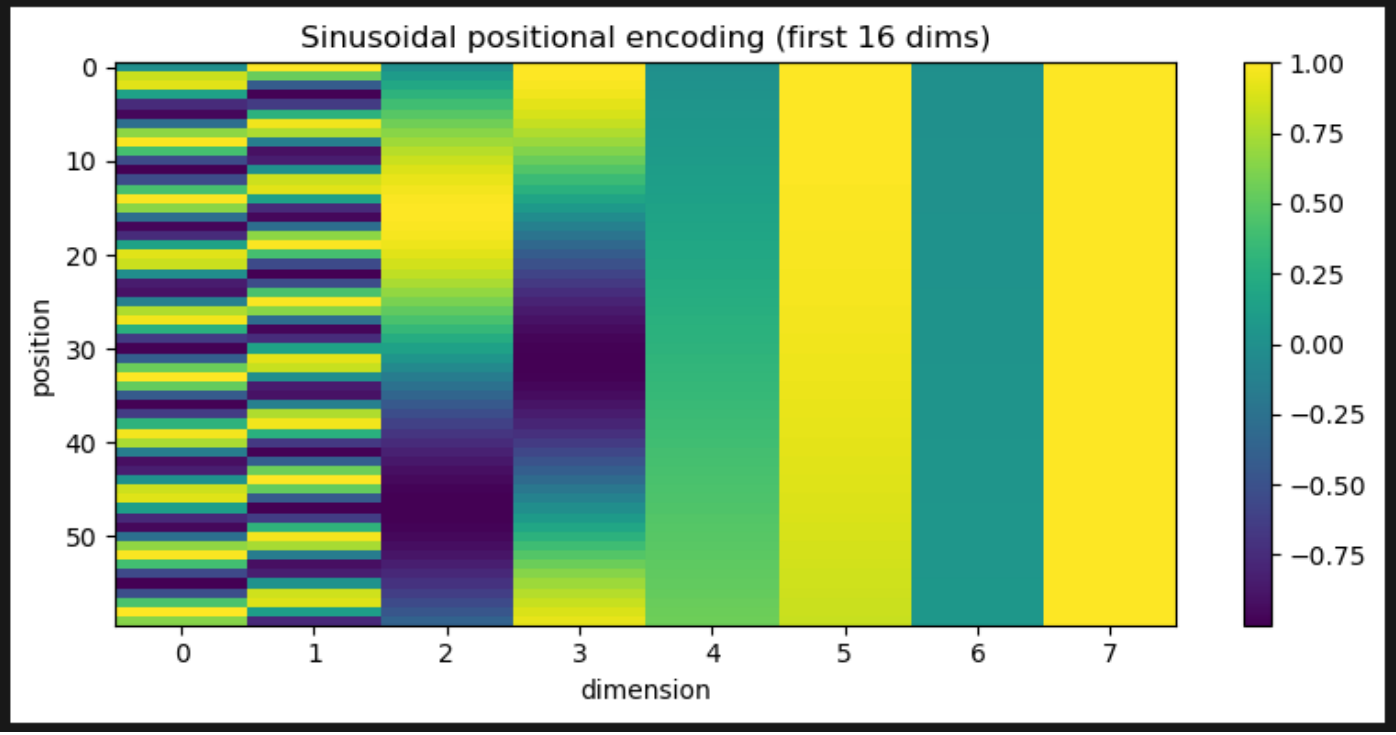
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension.

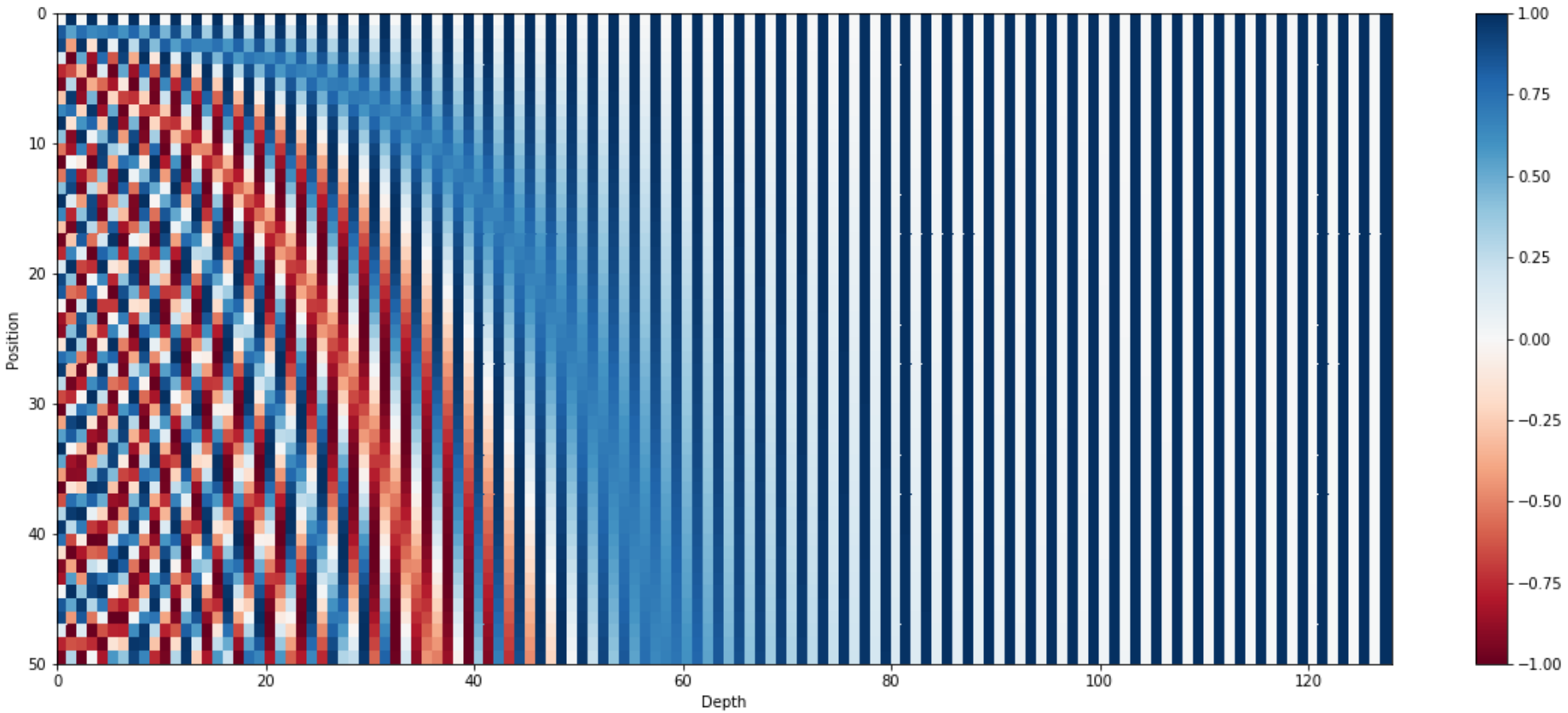
https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Positional Encoding

```
Position 0: [0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0]
Position 1: [0.8415, 0.5403, 0.0998, 0.995, 0.01, 0.9999, 0.001, 1.0]
Position 2: [0.9093, -0.4161, 0.1987, 0.9801, 0.02, 0.9998, 0.002, 1.0]
Position 3: [0.1411, -0.99, 0.2955, 0.9553, 0.03, 0.9996, 0.003, 1.0]
```



Positional Encoding



https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Characteristics of Sinusoidal Encoding

- Relative Positional Relationships
  - Allow the model to implicitly understand the relative distances between tokens.
  - $PE_{pos+k} = W \cdot PE_{pos}$ where W is not related to pos

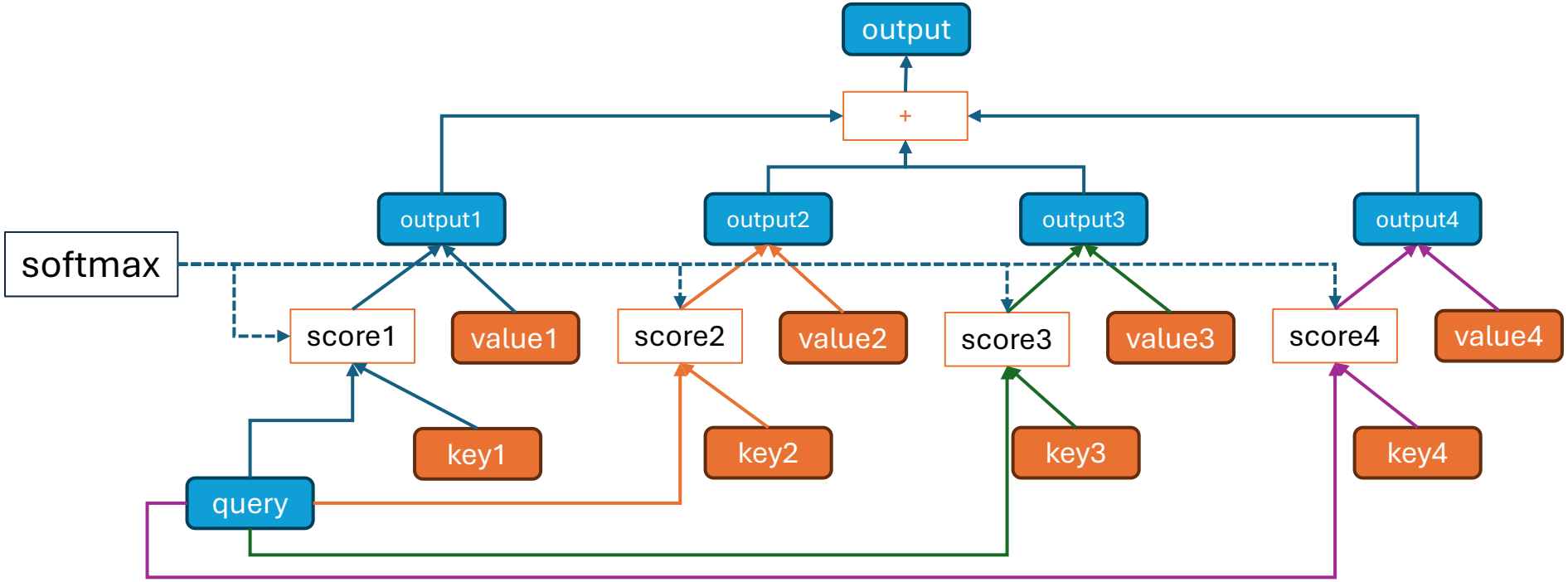
- Fixed and Non-learnable
  - The sinusoidal encoding is pre-defined and **does not require training**, making it computationally efficient and capable of generalizing to unseen sequence lengths.

Positional Encoding

- Different positional methods aim to improve:
  - Generalization to longer sequences
  - Modelling of relative positions
  - Stability and extrapolation
    - Can the model handle positions it has never seen during training?
  - Efficiency and numerical behavior
    - Fast enough to compute
    - Numerically stable for long or unseen positions.

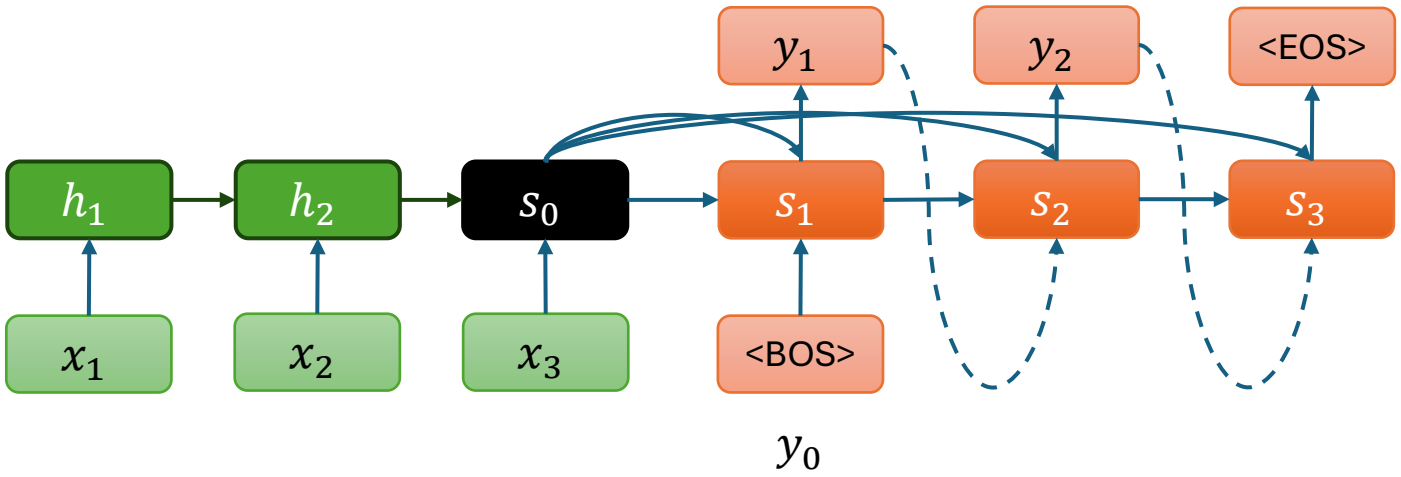
Review

- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
  - mapping a query and a set of key-value pairs to an output
  - output is computed as a weighted sum of the values
  - where the **weight** is computed by a **compatibility function of the query with the corresponding key**.



Review

- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$



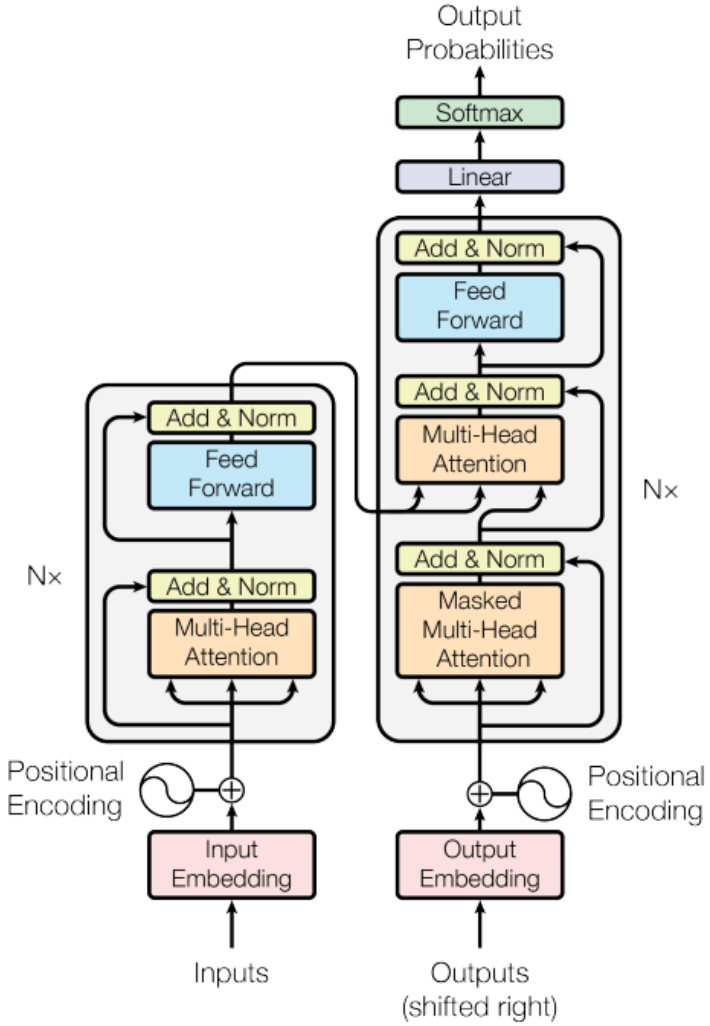


Figure 1: The Transformer - model architecture.

Review

- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- No future information
  - When calculating the current output, future information cannot be used, as it violates the auto-regressive property in sequence generation.
- Positional representation
  - Each position in the sequence is represented using a vector, rather than a single number, to encode positional information effectively.
- Layer normalization
- https://github.com/pytorch/pytorch/blob/main/torch/nn/modules/transformer.py
- https://github.com/pytorch/pytorch/blob/dc4db95540da06623c747bf0f2bf9f4af3d2925a/torch/nn/functional.py#L4026

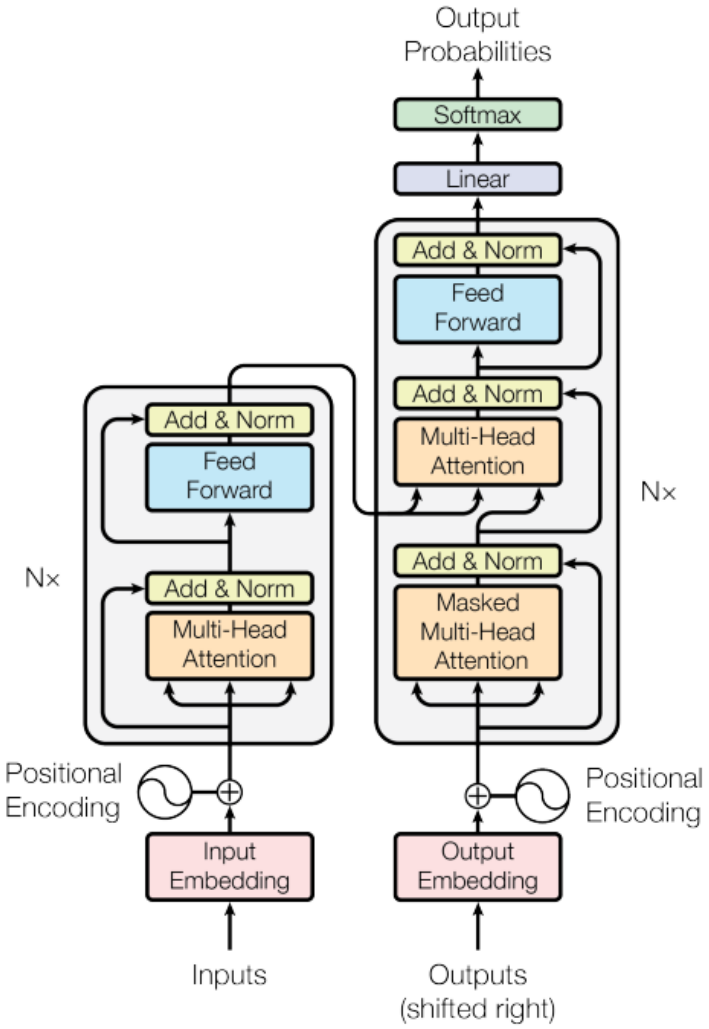


Figure 1: The Transformer - model architecture.

Learning Outcomes

- Explain the roles of query, key, and value in the attention mechanism
  - Demonstrate a clear understanding of how query, key, and value interact within the attention function to **compute context-aware outputs**.
- Deeply understand the concept of **autoregressive** modeling
  - Explain why the decoder in an autoregressive model **cannot access future information** and how this is enforced in practice.
- Analyze the Transformer architecture
  - **Identify and explain each module** in the Transformer network architecture diagram, including the encoder, decoder, attention layers, positional encoding, layer normalization, and feedforward components.